



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

1 - “Introduction to Computer Security”

Gabriele D'Angelo <g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

1 - “Introduction to Computer Security”

This block of slides **does not refer**
to a specific chapter in the book.

It is a “gentle introduction”

Gabriele D'Angelo <g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

A few **basic principles** of computer security:

- principle 1: "*non-existence of secure systems*"
- principle 1 (**corollary**): "*systems complexity*"
- principle 2: "*entities composing a system*"
- principle 3: "*security = knowledge*"

Security of a **cryptosystem** (*i.e., security through obscurity*)

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

- **The software is often far from perfect.** It is full of **implementation bugs** and **design errors.** It is the same for **communication protocols**
- **The “unbreakable system” is only a myth.** Just like the impenetrable bank vault or the unsinkable boat (e.g., *RMS Titanic*)

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

An attacker will attack a system
if there is enough value to
make the attack worth it

Security is costly: so there are various level of security.
It is very difficult to accurately assess the level of security required for a system.

The **security level** of a given system is determined by the **amount of**

(amount of time it takes an attacker to find a vulnerability,
develop an exploit, and breach the system)

(How much does it cost an attacker to
breach the system?)

time necessary to break the system, the **amount of money** (or

resources) and the **probability of success**

of the attack

Vulnerabilities can be discovered
(by a paid individual or a criminal)

Drastically reducing the probability of success (e.g., by
implementing two-factor authentication) discourages attackers,
because they know they will have to make a huge number of
attempts (increasing both time and costs) before succeeding.

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

How much am I willing to invest in identifying and addressing vulnerabilities in my system?

SECURE SYSTEMS DO NOT EXIST

Assets & Threat Modeling

What Are We Protecting? (The Assets)

- Hardware & Infrastructure: Servers, network devices, and cloud computing resources.
- Software & Services: Source code, applications, operating systems, and configurations.
- Data & Information: User credentials, financial records, intellectual property, and personal data (e.g., GDPR-regulated info).

- The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

What are we protecting?

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

Threat Actors

From Whom? (The Adversaries)

- Script Kiddies: Amateurs using pre-made tools for fun or notoriety.
- Cybercriminals: Well-organized professionals driven purely by financial gain.
- Insider Threats: Disgruntled, malicious, or negligent employees and partners.
- Hacktivists: Groups attacking systems to promote a political or ideological agenda.
- State-Sponsored Actors (APTs): Highly funded nation-state hackers focused on long-term espionage and sabotage.

- The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

Understanding who the enemy is, is important for determining the appropriate level of security.

What are we protecting? From **who**?

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

Motivations of an Attack

Why? (The Reasons for an Attack)

- Financial Gain (Money): Ransomware extortion, direct theft of funds, or unauthorized crypto-mining (Cryptojacking).
- Espionage: Stealing trade secrets, patents, or state intelligence for a strategic/competitive advantage.
- Sabotage & Warfare: interruption of critical infrastructure (power grids, transit) or destroying a competitor's capability.
- Ideology & Ego: Damaging a target's reputation (Defacement) or gaining prestige within hacker communities.

- The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

What are the
reasons for a
possible attack?

What are we protecting? From **who?** **Why?**

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Think in terms of value because that is the right way to assess the system's security level

SECURITY EXIST

- The **security level** is the **amount of** **time** necessary for an **attacker** to compromise the system (or resources) and the **cost** of the attack
- Most times the **attacker** adversary is rational
- Most times the adversary has an **economic goal** (that is... \$\$\$)
- This is **not always true** when the adversary is a **government**

What are we protecting? From who? Why?

The Proactive Approach (Threat Modeling): Take action before a security incident occurs. You put yourself in the attacker's shoes while the system is still in the design phase (on paper or in early prototypes) to identify potential vulnerabilities and address them before writing the final code.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Principle 1: on “Secure Systems”

- This is an “informal way” for defining a

THREAT MODEL

By defining the threat model, I can determine the level of security the system requires

↑ crates

- Threat modeling is a structured process
used in cybersecurity to identify and
analyze potential threats and
vulnerabilities in a system or application

DO NOT EXIST

The Core Objectives:

- Understand the system's architecture and attack surface.
- Identify what assets are targeted and what could go wrong.
- Define specific countermeasures (mitigations) before code deployment.

determined by the **amount of**

the **amount of money** (or

success

What are we protecting? From who? Why?

ZERODIUM Payouts for Mobiles*

Price tags

Up to \$2,500,000

Up to \$2,000,000

Up to \$1,500,000

Up to \$1,000,000

Up to \$500,000

Up to \$200,000

Up to \$100,000

Vulnerability Broker: collects vulnerabilities discovered by researchers and sells them (entities that buy and sell information about software vulnerabilities—usually zero-days—from security researchers, acting as intermediaries.)

FCP: Full Chain with Persistence
RCE: Remote Code Execution
LPE: Local Privilege Escalation
SBX: Sandbox Escape or Bypass

■ iOS
■ Android
■ Any OS

chain of vulnerabilities to gain root privileges (e.g., vulnerabilities in WhatsApp and the kernel)

When the device is restarted, the system remains compromised.

Zerodium purchases undisclosed cybersecurity vulnerabilities (known as zero-days) from independent researchers and hackers around the world, paying them handsomely. Zerodium then resells this information and the associated attack methods (exploits) to its select clients, which are primarily government agencies, intelligence services, and law enforcement agencies (for espionage or cyberwarfare purposes).

The price paid by Vulnerability Broker to purchase the vulnerability from researchers

3.001 Persistence iOS	2.005 WeChat RCE+LPE iOS/Android	2.006 iMessage RCE+LPE iOS	2.007 FB Messenger RCE+LPE iOS/Android	2.008 Signal RCE+LPE iOS/Android	2.009 Telegram RCE+LPE iOS/Android	2.010 Email App RCE+LPE iOS/Android	4.001 Chrome RCE+LPE Android	4.002 Safari RCE+LPE iOS
5.001 Baseband RCE+LPE iOS/Android	6.001 LPE to Kernel/Root iOS/Android	2.011 Media Files RCE+LPE iOS/Android	2.012 Documents RCE+LPE iOS/Android	4.003 SBX for Chrome Android	4.004 Chrome RCE w/o SBX Android	4.005 SBX for Safari iOS	4.006 Safari RCE w/o SBX iOS	
7.001 Code Signing Bypass iOS/Android	5.002 WiFi RCE iOS/Android	5.003 RCE via MitM iOS/Android	6.002 LPE to System Android	8.001 Information Disclosure iOS/Android	8.002 [k]ASLR Bypass iOS/Android	9.001 PIN Bypass Android	9.002 Passcode Bypass iOS	8.003 Touch ID Bypass iOS

code that runs automatically without user interaction

The chart shows how much money (in dollars) Zerodium is willing to pay for a single working exploit targeting mobile devices (Android and iOS). Prices vary based on two key parameters: the target (horizontal) and the type of attack (vertical). On the vertical axis, we find the severity/potency of the attack: RCE (Remote Code Execution): This is the most devastating attack. It allows the attacker to execute malicious code on the victim's phone remotely, without having physical access to the device. LPE (Local Privilege Escalation): The attacker already has limited initial access to the device and exploits this vulnerability to obtain maximum system privileges (e.g., Root or Administrator permissions). SBX (Sandbox Escape): Allows the attacker to break out of the "security sandbox" in which apps run in isolation to access data from other applications. FDC (Full Chain with Persistence): A combined attack that merges RCE, SBX, and LPE, allowing the phone to be completely and permanently compromised. On the horizontal axis are the most popular applications and operating systems: Operating systems: iOS (Apple) and Android (Google). Messaging apps: WhatsApp, Signal, iMessage, Telegram, WeChat.

* All payouts are subject to change or cancellation without notice. All trademarks are the property of their respective owners.

ZERODIUM Payouts for Mobiles*

Up to
\$2,500,000

Up to
\$2,000,000

Up to
\$1,500,000

Up to
\$1,000,000

Up to
\$500,000

Up to
\$200,000

Up to
\$100,000

FCP: Full Chain with Persistence
RCE: Remote Code Execution
LPE: Local Privilege Escalation
SBX: Sandbox Escape or Bypass

■ iOS
■ Android
■ Any OS

While RCE allows attackers to execute commands from a remote location to a device, LPE allows a low-privileged user to gain administrative or root-level access.

A full-chain vulnerability on Mobile is a sophisticated cyberattack that links together multiple individual software flaws, often zero-day vulnerabilities, to achieve complete, unauthorized control over a device. While a single vulnerability might only cause an app to crash, a full chain links, for example, a browser exploit, a sandbox escape, and a kernel privilege escalation to move from an external attack to full "root" or kernel-level control.

Is not a full-chain vulnerability

WhatsApp RCE example:

CVE-2022-36934

Exploit Whatsapp to be able to run unauth code on the device and then doing something on the device itself (maybe there is something after that maybe another privilege escalation)



* All payouts are subject to change or cancellation without notice. All trademarks are the property of their respective owners.

2019/09 © zerodium.com

Idea:
Exploit a
Vulnerability in the
Application and then
exploit a Kernel
Vulnerability to take
control of Root
Privileges



Zerodium ✓ @Zerodium · Jun 1

We're looking for #0day exploits affecting Pidgin on Windows and Linux.

Bounty: \$100,000

Read more: zerodium.com/temporary.html



9



129



213



Gary Kramlich

@rw_grim

Most of the time, the attacker is rational

The developer can introduce the vuln. and then selling them

A lot of software products rely on open-source libraries

Replying to @Zerodium

This really shows the sad state of funding in open source software.. I managed to find 25k USD last year to work on @impidgin full time, but if you can find a security exploit in my and other's non paid work you can make 4 times as much as I did... 🤔

9:59 PM · Jun 1, 2021 · Twitter Web App

Idea Cloud Computing:
Exploit App Vulnerability, Exploit
Virtual OS Kernel Vulnerability
and Exploit Virtualization
Software Vulnerability (to escape
the VM) to arrive on the HW or
OS of the Machine

Vuln. Broker are against the Fix of the
Vulnerability (the Manufacture wants to fix it)

So, a researcher that has found a
vulnerability can:
1) Sell the Vuln to Manufacture (Participate
to Bug Bounty Program: get way less
money than from Vuln. Brokers)
2) Sell the Vuln to Vuln. Broker

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

As a system grows in size and functionality, maintaining a high level of security becomes exponentially harder.

Why Security Fails in Complex Systems:

- Larger Attack Surface: More features mean more potential entry points for attackers.
- The Human Limit: No single engineer can understand the entire codebase and its dependencies.

• (**Empirical rule**) **Increasing the complexity** of a systems leads to an
insecure system

A linear increase in features/code results in an exponential increase in insecurity.

Interaction Flaws: Vulnerabilities rarely emerge from a single isolated component; rather, they arise from complex and unexpected interactions between multiple secure components.

• **Linear increase** of complexity means **super-linear increase** of
insecurity

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- Do you want to build a “**reasonably secure**” system?

Reduce the Complexity
of the System

KISS RULE:

→ avoid unnecessary complications.

- **Keep It Simple and Stupid**

- *Keep It Simple, Stupid*
- *Keep It Short and Stupid*

If you don't understand the
whole system, it is very difficult to
have that system secure

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- Does this mean that the systems to be secured can **not** be **complex** or **complicated**?

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- Does this mean that the systems to be secured can **not** be **complex** or **complicated**?

Can Modern Systems Be Complex? YES!

- High functional complexity is inevitable in modern environments (e.g., distributed systems).
- The Golden Rule: While the underlying system might be complex, the protection system must remain as simple as possible.

NO!



Principle 1: on “Complexity of Systems”

Example:

- Complex Scenario (Wrong): To secure access to servers, you create a custom proprietary system with 5 different encryption steps, tokens that expire every 12 seconds, which requires three different databases to verify identity, and which changes rules depending on the time of day.

> Result: A nightmare to manage. A minor bug in any of these steps will block legitimate users or create a huge logical flaw that a hacker will exploit.

- Simple Scenario (KISS - Correct): To secure the exact same complex server, use a standard, open, and widely tested protocol like SSH with public/private cryptographic keys and simple two-factor authentication (2FA).

> Result: The mechanism is straightforward, clean, very easy to verify, and leaves very little room for human error.

MORE COMPLEXITY → LESS SECURITY

It is the most important part of the whole system (example: Authentication of the device to use it)

The **protection system** must be “*as simple as possible*”. If possible...

“*very simple*”

- **Unnecessary complexity** can lead to {**design, implementation, usage**} errors

Unnecessary layers introduce a high risk of errors across three dimensions:

- Design Errors: Over-engineered security logic leading to structural blind spots.
- Implementation Errors: Hard-to-read, overly verbose code that conceals hidden bugs.
- Usage Errors: Complicated workflows that confuse administrators or force users to bypass security controls to remain productive.

- Each **error** is a potential “**vulnerability**”

Result: Every added complexity -> Potential Error -> Exploitable Vulnerability.

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

→ Zones of the system where we can have a Vulnerability.

A system consists of three interconnected components, and its overall security depends on the weakest link in the chain.

(MOST) SYSTEMS ARE COMPOSED OF 3 TYPES OF ENTITIES

- **Software** This is the logical component of the system. It includes everything that consists of code
- **Hardware** It is the physical, tangible component on which the software runs. (Physical security is guaranteed if the attacker cannot get their hands on the hardware)
- **Humanware** Humanware represents the people who interact with the system: users, network administrators, developers, managers, and customers.

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The attack surface is the set of all points where an unauthorized user (the attacker) can attempt to insert data, extract data, or interact with the system to find a vulnerability. Modern software has a massive attack surface because it consists of:

- User interfaces and APIs: Login forms, search bars, web endpoints. Every text field where a user can type something is a potential attack vector (e.g., if left unchecked, it can lead to SQL injection or cross-site scripting).
- Open network ports and protocols: Services listening to exchange data (HTTP, FTP, SSH).
- Third-party code (dependencies): Open-source libraries that developers import so they don't have to rewrite everything from scratch. If one of these external libraries has a vulnerability (as happened with the famous Log4Shell case), the entire application becomes vulnerable, even if the code you wrote is flawless.

SOFTWARE

- Example of a software attack (Difficult): Spend weeks analyzing the binary code of a banking application, find a memory bug (buffer overflow), write a complex exploit to bypass the operating system's defenses, and inject malicious code. This requires extraordinary skills and a great deal of time.
- Example of an alternative attack (Easy): Send a targeted phishing email to the bank's administrator (Humanware attack) pretending to be the director requesting an urgent check, prompting them to enter their credentials on a clone site. Within 10 minutes, the hacker is inside with full privileges, without having touched a single line of the banking software's code.

It is a critical part of the system with a **large exposed attack surface**

The software is critical and full of flaws, but remember that it's often much easier and quicker for a hacker to attack other components of the system (the hardware or, above all, the human element) rather than focusing on the code.

remember that... **often other parts of the system are “easier to attack”**



- Many, very complex programs
- Each program made of thousands or millions of lines of source code
(i.e. computer instructions)

- It is a critical part of the system with a **large exposed attack surface**
- remember that... **often other parts of the system are “easier to attack”**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

Air-Gapped Computer: This refers to a computer system that is physically isolated from all insecure networks, including the internet. The name implies there is a "gap of air" between the machine and any network, offering maximum security against remote, network-based, or malware attacks.

- It is a critical part of the system with a **large exposed attack**

S

That can be **accessed remotely** (i.e. from the Internet, from a wireless network or from a user interface)

(example: cars)

the system are “**easier to**

attack

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

doing a well attack surface
analysis isn't so easy

- It is a critical part of the system with a **large exposed attack**

surface

tech or not (example: a paper document containing information about a company is considered a significant part of an organization's physical attack surface)

- remember ***Everything*** that can be used by an system are “**easier to**
attack” **attacker to violate the system**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

- It is a critical part of the system with a **large exposed attack surface**
- remember that... **often other parts of the system are “easier to attack”**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Attacking the software often
requires some technical skills...

attacking the human in the
system requires skills that are not

about computing (psychological skills.
Example of attack: phishing)

SOFTWARE

system with a large exposed attack

- remember that... often other parts of the system are “easier to

attack”

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

When it is not possible to update the firmware to fix vulnerabilities, organizations must shift from applying patches to implementing mitigation measures, such as removing the hardware or reducing its attack surface, to manage the risks. Firmware vulnerabilities have a significant impact because they operate below the operating system level, often allowing persistent and invisible access.

HARDWARE

● All the modern hardware is composed of **hardware AND software**

(that is called firmware)

Firmware is a specific type of software that is built directly into a hardware component. Firmware can contain bugs and vulnerabilities just like standard software.

- The hardware can be **tampered** by an attacker

Examples: Inserting hardware keyloggers, stealing unencrypted hard drives, or exploiting supply chain vulnerabilities by altering components before delivery.

- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is hard to guarantee

If an attacker has unrestricted physical access to a device, it can no longer be trusted. Why it is hard to guarantee: Modern infrastructure is decentralized, making physical perimeters insufficient to protect all assets.

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HARDWARE

- All the modern hardware is composed of **hardware AND software** (that is called **firmware**)

- This means that an attacker can modify the hardware or ^{compromise} act on the software (e.g. flashing a modified firmware)

if the firmware is updatable and not well secured, the attacker can install a firmware with a backdoor

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

To interfere with a system (by touching it, altering it, or being in the middle) with the intent to cause harm or make unauthorized changes.

Manomettere/Alterare

Definition: “interfere with (something) in order to cause damage or make unauthorized ^{changes} alterations”

ARE

Because tampering is a huge risk, engineers design sensitive systems using two defensive approaches:

- Tamper-Resistance: Creating physical or logical barriers that make it extremely difficult to alter the system.
- Tamper-Evident: If I cannot prevent a hacker from accessing the system, I ensure that any tampering is immediately visible. In the physical world, security seals (sigilli di sicurezza) are used (stickers that break if you try to open them); in the software world, encryption is used (e.g., hash functions or digital signatures) to instantly detect if even a single bit of a file has been altered.

composed of **hardware AND software**

physical tampering with a device's components, circuits, or logic for malicious purposes. Tampering requires that the attacker have physical access to the hardware.

- The hardware can be **tampered** by an attacker →
- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is hard to guarantee

How can you know if a hardware device has been tampered with? To ensure that a hardware device has not been tampered with, you must ensure the physical security of the device (since, if a hardware device has been tampered with, it is very difficult to detect).

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HARDWARE

The “evil maid attack” is an example of unauthorized alteration of personal devices

composed of **hardware AND software**

- The hardware can be **tampered** by an attacker
- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is hard to guarantee

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Possible to put a backdoor in a binary firmware without knowing the source code (it easily to place it in the middle of the firmware)

To prevent unauthorized modifications, modern CPUs rely on hardware-based tamper-resistance, which cryptographically signs and verifies the firmware before execution.

HARDWARE

The Physical security of an HW must be guaranteed across the full hardware lifecycle, from initial purchase to final disposal (because if control were lost even for just a few seconds, it could be tampered). We do this because it is very difficult to detect if the HW it was tempered

It is an attack that exploits temporary physical access to compromise a device so that the attacker can control it remotely (by inserting a backdoor into the device's firmware).

The “**evil maid attack**” is an example of
unauthorized alteration of personal devices

Evil Maid Attack Demo

A “**nice demo**”: the time required
to flash the tampered firmware is
very limited

- The hardware can be **tampered** by an attacker
- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is **hard to guarantee**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Lack of knowledge: The user or administrator simply does not know that a certain action is dangerous (e.g., reusing the same password for work and personal websites).
- Overload & Fatigue: A tired employee, overwhelmed by countless emails and deadlines, has a much lower attention span. It is in that moment of fatigue that they will click on the phishing email without checking it carefully.
- Bad design: If a security system is poorly designed, frustrating, and too complicated to use, the user will try to bypass it just to get their work done.
- Goodwill: Humans are inclined to help others. A hacker can exploit this “goodwill” by posing as a colleague in trouble who urgently needs a file or access to avoid losing a client.
- Lack of attention: A one-second distraction can lead to sending a confidential document to the wrong recipient.
- Stupidity (in both users and administrators): attitudes of total laziness or unjustifiable negligence. For example, a user who deliberately ignores antivirus security warnings, or a system administrator who leaves the main server password as “admin/admin” or “123456.”

HUMANWARE

No matter how secure the hardware or software is, humans remain the primary target for attackers.

- The “**human in the loop**” is often the **weakest link in the system**

(e.g. **social engineering**)

Attackers exploit human psychology (fear, urgency, trust) rather than technical flaws to bypass defenses (e.g., Phishing).

- For **many reasons**: lack of knowledge, overload, fatigue, bad design, goodwill, lack of attention... stupidity (*in both **users** and **system administrators***)

Principle 2: on “Entities

Social engineering is widely recognized as the most common and effective method for cyberattacks in the history of cybersecurity. According to industry statistics, social engineering techniques are involved in 98% of cyberattacks, making the “human element” the weakest link in security.

Definition: “^{inganno}the use of deception to manipulate ^{to push him/her}individuals into divulging confidential or ^(that goes against cybersecurity policies)personal information that may be used for fraudulent purposes” (e.g. phishing emails)

TER STUDIORUM
ITÀ DI BOLOGNA

- The “**human in the loop**” is often the **weakest link** in the system (e.g. **social engineering**)

- For **many reasons**: lack of knowledge, ^{sovraccarico (può essere indotto dall'attaccante)}overload, fatigue, bad ^{of tools}design, goodwill, lack of attention... stupidity (in both **users** and **system administrators**)

In such cases, people tend to take shortcuts by not following cybersecurity policies, which makes the system vulnerable

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HUMANWARE

Correct way to mitigate this problem is through procedures (properly designed) to identify if I'm talking with a colleague or attacker

- The “**human in the loop**”
(e.g. **social engineering**)

buona volontà
Is “goodwill” a problem?

(example: faking to be a colleague and ask for confidential info)

Yes, when abused by attackers

- For **many reasons**: lack of knowledge, overload, fatigue, bad design, goodwill, lack of attention... stupidity (in both **users** and **system administrators**)

We are stupid when we don't understand something

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Operational Security (OPSEC) is the process of protecting sensitive information by looking at your own operations through the eyes of an attacker.

SECURITY AS A PROCESS

Cybersecurity isn't a product

- Securing a system is a [**never ending**] **process** and **not a product**

(Cybersecurity is a day-by-day investment)

- **Reasonably secure system** requires **an endless work** on the system and **education**

secure

- 1 • The system that today is “secure”, tomorrow will not be. **Everyday news faults are discovered**

- 2 • The **lack of updates** leads to **fragile** and **insecure** systems (system can be compromised)

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SysAdmin focus on Updates (when to do it, how, etc..)

EVEN UPDATING IS NOT EASY...

- Are the updates **available** for all your devices? (e.g. **smartphone**)
- In other words, **is your devices supported? Until when?**

→ if the device is no longer supported, the device is vulnerable if they discover a vulnerability and you cannot update it

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

EVEN UPDATING IS NOT EASY...

- Are the updates **available** for all your devices? (e.g. **smartphone**)
- In other words, **is your devices supported? Until when?**

Based on
Company
Context



Update Policy



If the updates are available...

The tech guy need to define the Update Policy

- **what** update to apply? (**All of them...** are you sure of that?) Choose which part of the update to do
- **when** do you apply updates? (During the **night**? In “**production**”?)
- **why**? (Is that for **security** or to add **new functionalities**?)
- **how**? (What is the best way to apply the update? Automatic or **Unattended**?)

if the vuln. is public known, update ASAP, considering the time (example: customer peeks, etc..)



Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

EVEN UPDATING IS NOT EASY...

- The **updating process** is always a dangerous operation
 - for example: fixing bugs can introduce new bugs, instability, side-effects... due to complexity of the system we are dealing with
- What we need is a **costs/benefits evaluation**
 - that is **costly** (e.g. time consuming)
 - it is often based on **partial information** to take decision on a specific update (i.e. often there is a lack of detailed information on the **updates content**)

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

Trust is even more
important than Knowledge

- Real security is based on **deep knowledge** (of the system to be protected)
- What are the **effects** of this principle **on software**?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- Real security is based on **deep** (not just superficially protected) **knowledge** of the system (which must be **shared**)
- What are the **effects** of this principle on software?
 - **Open Systems vs. Closed Systems**
 - **Open Source vs. Closed Source**

The specifications of the system are

OPEN / CLOSED

I really prefer **OPEN** specifications (e.g. for interoperability^{with other system}) but this does **not** guarantees that the system is secure (e.g. due to design or implementation faults)

Knowledge”



E OF THE SYSTEM

- Real security (based on **dec** protected)

The **specifications** of the system are
OPEN / CLOSED

If they have specs open/closed, doesn't guarantee security by definition

- What are the **effects** of this principle on software?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- Real security is based on **deep knowledge** (of the system to be protected)
- What are the **effects** of this principle **on software**?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- Real security is based on **knowledge** (if the system is protected)

The source code (i.e. computer instructions) used by the programmers to implement the system are

OPEN / CLOSED

doesn't guarantee security by definition

- What are the **effects** of this principle on software?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Knowledge”

I really prefer **OPEN** Source (e.g. it can be *inspected, verified, validated* etc.) but this does not guarantees that the system is secure

GE OF THE SYSTEM

The **source code** (i.e. computer **instructions**) used by the programmers to implement the system are

OPEN / CLOSED

- Real security is based on **data** protected
- What are the **effects** of this principle on software?
 - **Open systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**



I really prefer **OPEN** Source (e.g. it can be *inspected, verified, validated* etc.) but this does **not** guarantee that the system is secure

Even if it is theoretically possible (for me) to check all the code in a system (e.g. an open source Operating System like GNU/Linux), this is not practically possible (since it is made of millions of lines of source code)

Even if I have the source code of the program, it is difficult to know if the binary version of it contains a backdoor or malicious code that is different from the source code that is public available

- Real security is based on protected code
- What are the effects of this principle on software?
 - Open systems vs. Closed Systems
 - **Open Source** vs. Closed Source

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Management of Trust is fundamental in Cybersecurity and based on a lot of different stuff (example: choosing the browser based on history of vuln. fixing)

KNOWLEDGE OF THE SYSTEM

- Real security is based on knowledge (of the system to be protected)

- What are the **effects**

- **Open Systems**

- **Open Source v**

In most cases it is a matter of

Security is based on Knowledge about the system we want to secure

→ baseline of CySec (define the allies and enemies)

TRUST

→ more important than knowledge (because it is very difficult to know/verify/check every single detail of the system)

Do you prefer to trust a

community (i.e. Open Source) or

a company (i.e. Close Source)

The future in terms of guaranteeing the supply-chain is a game not based on verifying everything, it will be based a lot on certifications (made in-house or by third-party certifiers) to demonstrate to customer that you are implementing cybersecurity

Nobody, now, create SW that is totally developer e/o designed in house (we use a lot of library, sw made by others, etc.): possible problem of the supply chain (example: (supply-chain attack) possible backdoor in our program that came from library developed by others)

Auto-Updates is a pro and cons:

- Pro: install automatically the fix for a detected vuln.
- Cons: install updates automatic with backdoors or other vuln.

Product of a Manufacture is based on Software supply chain (built on libraries, services, etc.): if something bad happens the responsibility is on the Manufacture (this is why they need to certificate them to build trust on Manufacture's product)

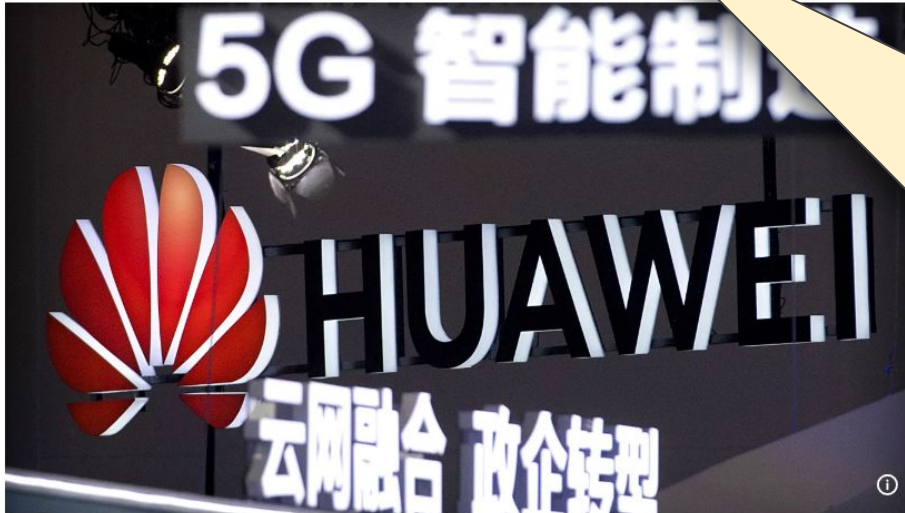
Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Eleven EU countries took 5G security measures to ban Huawei, ZTE

because they cannot go in deep on the software of these devices to check about vulnerabilities like backdoors (they restricted because they don't have trust on some companies: possible that China gather info through it (not verified))



«Eleven countries [...] have used legal powers to impose **restrictions** on telecom suppliers that are considered high-risk, such as Huawei and ZTE, for 5G network infrastructure, a European Commission spokesperson told Euronews»

Copyright Mark Schiefelbein/Copyright 2018 The AP. All rights reserved

By [Cynthia Kroet](#)

Published on 12/08/2024 - 16:37 GMT+2

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- **Users' education** is a form of **knowledge**
- **Investing in education** is good but **not that simple**:

- education is **costly**

It is important but a few cares
about it (increasing in these days)

- **perception of security**: [lack of] importance of cybersecurity

- [wrong] **habits** (i.e. “*we have always done it this way*”)

- **ideological resistance** (i.e. “*information anarchism*”) (example: don't use passwords)

to win this battles:
use of regulations



- **cost of security** procedures (vs. “*productive investments*”)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Principle Important to the Prof

KERCKHOFFS'S PRINCIPLE

- “A cryptosystem should be secure **even if everything about the system, except the key, is public knowledge**” (1883)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KERCKHOFFS'S PRINCIPLE

→ Algo that we use for encrypting the data, must be public (the only part that must be secret is the key, a small piece of information)

● “A **cryptosystem** should be secure **even if everything about the system, except the key, is public knowledge**” (1883) It is against Security through obscurity

Cypher (cifrario)

It is a mechanism (i.e. hardware or
algorithm
software) used for encrypting information
with the aim to preserve confidentiality

(encryption to preserve this property)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KERCKHOFFS'S PRINCIPLE

- “A **cryptosystem** should be secure **even if everything about the system, except the key, is public knowledge**” (1883)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KERCKHOFFS'S PRINCIPLE

- “A **cryptosystem** should be secure **even if everything about the system, except the key, is public knowledge**” (1883)

SHANNON'S REFORMULATION



“The **enemy knows the system**”

not every secret of the system, but the design of the system

It is a much better way to base the sec on the selection of the proper algo, not on secrecy of the algo (maintaining private the key)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURITY THROUGH OBSCURITY

Wrong: against Kerckhoffs
principle (algo secret)

- Means relying on the **design** or **implementation secrecy** as the main method of providing security

It is very difficult to maintain during the years

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURITY THROUGH OBSCURITY

- Means relying on the **design** or **implementation secrecy** as the main method of providing security
- The **security through obscurity** is **WRONG**

Addendum: "Security of a Cryptosystem"



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The **security through obscurity** is **WRONG**

- Does this means that I must **publish all the details about my system internals?**

When develop a new application, there are two separate path that you can choose to adopt encryption:

- 1) Use standards (Good Approach: study papers and choose the perfect one)
- 2) Create a new algo for encryption (Stupid Approach: will do something with a lot of vulnerabilities and errors, if you are not an expert)

The Core Flaws:

- Fragility: Once the "secret" is discovered, the security is 0. It has no backup plan.
- The "Script Kiddie" Problem: While it might hide you from basic automated bots, it won't stop a human attacker with basic tools like scanners or decompilers.
- Complacency: It often leads developers to ignore real vulnerabilities because they assume "no one will find it anyway."
- "Violates Kerckhoffs's Principle: True security should remain intact even if the attacker knows exactly how the system works, as long as they don't have your specific key.

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Ask why and what you
are going to secure with
this security mechanism
(define a model)

The security through obscurity is **WRONG**

- Does this mean that I must **publish all the details about my system internals?**

- **NO**, it would be very stupid!



will decrease the cost for an attacker

The security must not be based on secrecy, but this doesn't mean that you have to publish all details (you need to increase the cost for attacker to breach the system or gather info)

When we build up a security mechanism, we are totally losing common sense and avoid only cosmetic security mechanism: the security mechanism is useful if it is possible to enforce it (users will try to bypass it)

Before disclosing information, evaluate whether it is truly necessary. Excessive disclosure fuels OSINT (Open Source Intelligence), allowing attackers to gather the data needed to refine and launch a more effective attack.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

Gabriele D'Angelo

<g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

www.unibo.it

Attributions and Notes



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- These slides are partially based on the work of Prof. **Renzo Davoli**
- **Ludovico Gardenghi** has worked on a previous version of these slides
- I wish to **thank all of them**, **any errors that remain are my sole responsibility**